Assignment-1 : Regression for Electrical Engineering Problems

AI/ML for Electrical Engineering · Gati Shakti Vishwavidyalaya, Vadodara

Name	Anshul Pareek	**Roll no.**	24EE009
Due		**Total marks**	100
numpy version	2.1.3	**scikit-learn version**	1.6.1

Assignment Parts:

Please write your own scripts for every regression model studied and use them on six electrical-engineering datasets. The aim is not a high accuracy score; the aim is that you can say *why* a model behaved the way it did.

Part A	Linear regression four models, and what outliers do to least squares	15
Part B	Gradient descent written from scratch (NumPy only)	20
Part C	Polynomial regression, learning curves, and extrapolation	20
Part D	Ridge, Lasso and Elastic Net on many correlated signals	20
Part E	Logistic regression for fault detection	15
Part F	Softmax regression for power-quality events	10
Part G	Caught-GenAI-being-wrong note (required, ungraded)	—

How to submit

- This sheet, filled in, as a PDF (type into it or scan your handwriting — both are fine).
- One script per part: `partA.py` … `partF.py`. Each must run from a fresh terminal with the `data/` folder next to it and print every number you wrote on this sheet.
- The figures the tasks ask for, as PNG files named as stated.
- Upload all files to Google Classroom as one zip (contains scripts, PDF and PNGs).

GenAI: You may use GenAI tools, as per the course policy. You must fill in Part G. Fill the amber *Own Predictions First* boxes before running anything; they are marked for being present and reasoned, *not* for being right. A wrong prediction you can explain is worth more than a right one you copied.

Implementation rules (follow exactly, so that answers can be checked)

R1. Use Python with NumPy, pandas, Matplotlib and scikit-learn. **Part B: NumPy only** (no scikit-learn).

R2. RMSE = $\sqrt{(\ (1/m)\ \sum_i\ (\hat{y}_i - y_i)^2\ )}$, in the units of the target.

R3. Use the `split` column exactly as given. Never fit anything — model, scaler, or hyperparameter — on validation or test rows.

R4. "Standardise" means `StandardScaler`, fitted on the training rows only (Part B: `np.mean`, `np.std` with `ddof=0`).

R5. Cross-validation, wherever used: `KFold(n_splits=5, shuffle=True, random_state=42)`.

R6. Leave every other scikit-learn setting at its default unless the task says otherwise.

R7. Report numbers to the decimal places shown in each table. Small differences in the last digit from library versions are acceptable.

Folder `data/`)

All datasets are generated from physically motivated models with measurement noise. The true generating models will be released after the deadline so you can check your conclusions.

File	Rows	Columns (units)
`A_winding_resistance.csv`	25	`temp_C` (°C), `resistance_ohm` (Ω) — bench test of a copper winding
`A_winding_resistance_logger.csv`	28	same test, as recorded by the plant data logger
`B_pv_plant.csv`	500	`irradiance_Wm2` (W/m^2), `module_temp_C` (°C), `ac_power_kW` (kW); no split
`C_feeder_load.csv`	380	`temp_max_C` (°C), `peak_load_MW` (MW), `split`: train 60 / val 300 / `test_heatwave` 20
`D_motor_thermal.csv`	560	30 inputs: `i_band_01`–`i_band_20` (stator-current spectral band levels, p.u., in order of frequency), `speed_rpm`, `torque_Nm`, `ambient_C`, `coolant_temp_C`, `coolant_flow_lpm`, `aux_01`–`aux_05` (spare acquisition channels, nothing connected). Target `winding_rise_K` (K). `split`: train 60 / test 500
`E_bearing_condition.csv`	600	`vib_rms_mm_s` (mm/s), `kurtosis` (–), `fault` (0 healthy, 1 faulty), `split`: train 360 / val 120 / test 120
`F_pq_events.csv`	700	`rms_pu` (p.u.), `thd_pct` (%), `event` (normal, sag, swell, harmonic), `split`: train 490 / test 210

Part A — Linear regression four models [15 marks]

The DC resistance of a copper winding rises with temperature as $R(T) = R_{20}\,[1 + \alpha(T - 20)]$, where R_{20} is the resistance at 20 °C and α is the temperature coefficient of resistance. This is linear in T: $R = \theta_0 + \theta_1 T$. Your job is to estimate R_{20} and α from a bench test (`A_winding_resistance.csv`), then see what happens to the estimate due to a data logger's glitches.

Own prediction first — fill this in BEFORE you run any code

Write θ_0 and θ_1 in terms of R_{20} and α. Roughly what value do you expect for α (look it up for copper; cite your source)?

Expanding $R(T) = R_{20}[1 + \alpha(T - 20)] = R_{20}(1 - 20\alpha) + (R_{20}\alpha)T$ and matching with $R = \theta_0 + \theta_1 T$ gives $\theta_0 = R_{20}(1 - 20\alpha)$ and $\theta_1 = R_{20}\alpha$. For standard annealed copper at 20 ˚C, the expected temperature coefficient of resistance is $\alpha \approx 0.00393\ ^\circ\mathrm{C}^{-1}$
(IEC 60028 / CRC Handbook of Chemistry and Physics).

A1. Fit $R = \theta_0 + \theta_1 T$ in four ways. (a) By hand with NumPy sums: $\theta_1 = S_{xy} / S_{xx}$, $\theta_0 = \bar{y} - \theta_1 \bar{x}$. (b) Normal equation with `np.linalg.inv`. (c) `np.linalg.lstsq`. (d) `LinearRegression`. **[4]**

Method	θ_0 (Ω, 6 d.p.)	θ_1 (Ω/°C, 8 d.p.)
(a) S_{xy}/S_{xx} by hand	0.783905	0.00333176
(b) Normal equation	0.783905	0.00333176
(c) `np.linalg.lstsq`	0.783905	0.00333176
(d) `LinearRegression`	0.783905	0.00333176
Largest absolute difference between any two methods		1.5543×10^{-15}
Training RMSE of the fit (Ω, 5 decimal points(d.p.))		0.00176

A2. From your fit, compute $R_{20} = \theta_0 + 20\,\theta_1$ and $\alpha = \theta_1 / R_{20}$. **[3]**

R_{20} (Ω, 4 d.p.)	0.8505 Ω
α (per °C, 6 d.p.)	$0.003917\ ^\circ\mathrm{C}^{-1}$
Handbook α for copper (with source)	$0.00393^\circ\mathrm{C}^{-1}$
Difference from handbook (%, 1 d.p.)	0.3%

A3. Now fit the logger file `A_winding_resistance_logger.csv` with `LinearRegression` and with `HuberRegressor` (defaults). Then list every row whose residual from the Huber fit exceeds 0.02 Ω in magnitude. **[4]**

Fit on logger data	R_{20} (Ω, 4 d.p.)	α (6 d.p.)	Change in α vs A2 (%)
`LinearRegression` (MSE)	0.8071	0.004725	+20.6%
`HuberRegressor`	0.8507	0.003915	−0.1%

Rows with |Huber residual| > 0.02 Ω:

temp_C	resistance_ohm	Huber residual (Ω)
35.0	1.0223	+0.121641
50.1	0.0000	−0.950950
72.4	1.1483	+0.123078

A4. Which single logger reading did the most damage to the least-squares fit, and why does squared error let it? Would you simply switch to the Huber fit in a commissioning report? What would you do instead? **[4]**

1. Damaging reading and squared-error penalty: Row 12 (T = 50.1 ◦ C,R = 0.0000Ω) caused by far the most damage. Its residual magnitude from the true curve is ≈ 0.9510Ω, compared to normal bench measurement noise of ≈ 0.0018Ω. Because Ordinary Least Squares minimizes ∑ ie2 i, an outlier with residual ei incurs an error penalty proportional to e2 i and applies a gradient pulling force of 2eixi. A single −0.951Ω dropout exerts (0.951/0.0018)2 ≈ 280,000 times the penalty weight of an inlier, heavily skewing the normal equations (shifting R20 down by 5.1% and inflating α by +20.63%).

2. Commissioning strategy: No, one must never silently substitute Huber Regressor in an official engineering commissioning report. Doing so masks severe physical hardware malfunctions (e.g., loose 4-wire Kelvin clips, shorted instrumentation cables, ADC bit-dropouts, or power supply brownouts). Instead:

1. Interrogate the raw logger timestamps and physical test conditions to diagnose the hardware root cause of the 0Ω reading.
2. Formally document, flag, and quarantine the defective records as verified sensor acquisition glitches.
3. Retrain standard OLS on the verified, clean physical dataset (using HuberRegressor only as an automated statistical diagnostic to flag suspect rows), reporting both the clean physical parameters and the acquisition system failure rate.

Part B — Gradient descent [20 marks]

NumPy only. Model: $\hat{P} = \theta_0 + \theta_1 G + \theta_2 T_m$ for the PV plant in `B_pv_plant.csv` (G = irradiance, T_m = module temperature). There is no train/test split in this part since this is to study the optimiser, not generalisation. Write $\mathbf{X}$ with a leading column of ones. The gradient of the MSE is $(2/m)\mathbf{X}^\top(\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$ and its Hessian is $\mathbf{H} = (2/m)\mathbf{X}^\top\mathbf{X}$.

Batch GD rules for this part. Start from $\boldsymbol{\theta} = \mathbf{0}$. Stop when $(J - J^\star)/J^\star < 10^{-6}$, where J is the current training MSE and $J^\star$ the MSE of the closed-form solution. Give up after 200,000 iterations. Consider an execution run *diverged* if J exceeds 10^6 times its starting value.

Own prediction first — fill this in BEFORE you run any code

Will gradient descent converge quickly, slowly or not at all while considering with raw units (G in hundreds, T_m in tens)? How many iterations do you guess for raw vs standardised inputs?

With raw units ($G \sim 10^2$–10^3 W/m^2, $T_m \sim 10^1$ °C, and intercept $= 1$), the Hessian is severely ill-conditioned, so batch gradient descent with $\eta = 1/\lambda_{\max}$ will converge extremely slowly and likely hit the 200,000-iteration cap before reaching the 10^{-6} relative MSE tolerance. On standardized inputs (unit variance, zero mean), the loss surface becomes nearly spherical, so gradient descent should converge in roughly a few hundred iterations (200–500 steps).

B1. Closed-form solution with `np.linalg.lstsq` on raw inputs. **[2]**

$\theta_0^\star$ (kW, 4 d.p.)	$\theta_1^\star$ (6 d.p.)	$\theta_2^\star$ (6 d.p.)	$J^\star$ (kW^2, 6 d.p.)
0.2116 kW	0.004929	-0.011111	0.009974 kW^2

B2. Hessian and batch GD with $\eta = 1/\lambda_{\max}(\mathbf{H})$, on raw inputs and on standardised inputs (standardise G and T_m only, not the column of ones). **[5]**

	Raw inputs	Standardised inputs
Eigenvalues of $\mathbf{H}$ (3 s.f. each)	0.0411, 183, 7.80×10^5	0.117, 2.00, 3.88
Condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ (3 s.f.)	1.90×10^7	33.2
Stability limit $2/\lambda_{\max}$ (3 s.f.)	2.57×10^{-6}	0.515
Iterations to converge (or "not converged")	Not converged (200000)	257
If not converged: $(J - J^\star)/J^\star$ at 200,000	0.0907	-

B3. Convert the standardised-input solution back to raw units and compare it with B1. Give the formulae you used and the largest absolute difference. **[2]**

$$\theta_1 = \frac{\theta_{1,\text{std}}}{\sigma_G}, \qquad \theta_2 = \frac{\theta_{2,\text{std}}}{\sigma_{T_m}}$$

$$\theta_0 = \theta_{0,\text{std}} - \theta_{1,\text{std}}\frac{\mu_G}{\sigma_G} - \theta_{2,\text{std}}\frac{\mu_{T_m}}{\sigma_{T_m}}$$

Converted coefficients: 0.21163146, 0.00492943, -0.01111141
Closed-form raw coefficients: 0.21163146, 0.00492943, -0.01111141
Largest absolute difference: 5.967×10^{-15}

B4. Learning-rate sweep on standardised inputs: $\eta = k \times (2 / \lambda_{\max})$. **[4]**

k	η (3 s.f.)	Status (converged / diverged)	Iterations (to converge or to diverge)
0.1	0.0515	converged	1300
0.5	0.258	Converged	257
0.99	0.510	Converged	469
1.01	0.520	diverged	390

B5. Mini-batch gradient descent on standardised inputs. Use exactly: `rng = np.random.default_rng(0)`; at the start of every epoch `idx = rng.permutation(m)`; batches `idx[i:i+32]` in order; step counter t starts at 0 and increases by 1 per batch; step size $\eta_t = 5/(t + 50)$. Report $\|\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{std}}^{\star}\|$ (distance to the standardised closed-form solution). Then try one learning schedule of your own and report it too. Save a plot of the distance against epoch as `B5_minibatch.png`. **[3]**

Schedule	after 1 epoch	after 5 epochs	after 20 epochs
$\eta_t = 5/(t + 50)$ (4 s.f.)	0.8778	0.5844	0.3163
Your schedule: 1/(t+10)	0.9666	0.7922	0.6800

B6. Explain, using the eigenvalues of ($\mathbf{I} - \eta\mathbf{H}$), why $k = 0.99$ converged and $k = 1.01$ diverged. Then explain in one or two sentences why standardising changed the raw-input result so dramatically.**[4]**

1. Stability via eigenvalues of ($I - \eta H$): In batch gradient descent, the parameter error vector $et = \theta_t - \theta^*$ evolves according to $e_{t+1} = (I - \eta H)e_t$. The eigenvalues of $(I - \eta H)$ are $\mu_i = 1 - \eta\lambda_i$. Convergence to zero error requires the spectral radius $\rho(I - \eta H) = \max_i |1 - \eta\lambda_i| < 1$, which strictly dictates $0 < \eta < 2/\lambda_{max}$. Here $\lambda_{max} \approx 3.882$, giving stability limit $\eta_{crit} = 2/\lambda_{max} \approx 0.5152$.

- When $k = 0.99$, $\eta = 0.99(2/\lambda_{max})$, the eigenvalue corresponding to λ_{max} becomes $\mu_{max} = 1 - 0.99(2) = -0.98$. Since all update-matrix eigenvalues have magnitudes below 1, every error component decays, and the algorithm converges in 469 iterations.
- When $k = 1.01$, $\eta = 1.01(2/\lambda_{max})$, that eigenvalue becomes $\mu_{max} = 1 - 1.01(2) = -1.02$. Because $|\mu_{max}| = 1.02 > 1$, the error component along the leading eigenvector is magnified by 2% on every iteration with alternating sign $((-1.02)_t)$, resulting in unbounded exponential oscillations and diverging at iteration 390.

2. Effect of standardization: On raw inputs, vast differences in feature scales warp the MSE cost surface into an extremely narrow, eccentric parabolic canyon with $\kappa \approx 1.9 \times 107$, forcing the step size to be infinitesimally small to avoid cross-canyon divergence. Standardizing centers and scales all features to unit variance, making the loss surface nearly isotropic (spherical contours), collapsing κ down to 33.2, and enabling large, uniform gradient steps across all directions simultaneously.

Part C — Polynomial regression and extrapolation [20 marks]

A distribution utility wants a model of daily feeder peak load against the day's maximum temperature, for summer planning. You have 60 training days and 300 validation days, all between 8 and 40 °C. A separate set of 20 days, `test_heatwave`, comes from a heatwave hotter than any training day. **Do not look at the heatwave rows until task C3.**

Polynomial model for degree d: `make_pipeline(PolynomialFeatures(d, include_bias=False), StandardScaler(), LinearRegression())`.

Own prediction first — fill this in BEFORE you run any code

Plot the training data first. Which degree do you expect validation to choose? Will the chosen model be safe to use on a 45 °C day? Why?

Plotting the training rows (8.2 °C to 38.3 °C) shows a U-shaped curve with a flat comfort deadband around 18–24 °C, mild heating load rise below 18 °C, and a steep quadratic cooling load rise above 24 °C. I expect the validation set to select a low-to-moderate polynomial degree ($d \approx 3$ or 4, or a higher degree that bends sharply to match the piecewise comfort deadband kinks). However, any pure high-degree polynomial chosen by validation will **not** be safe to use on a 45 °C day because polynomials are non-local functions whose highest-power term (T^d) dominates outside the 8–40 °C training interval, causing severe runaway oscillation or blow-up during extrapolation.

C1. Fit each degree on the training rows. Report training and validation RMSE. **[4]**

Degree d	1	2	3	4	6	10	15
Train RMSE (MW, 1 d.p.)	80.5	30.2	27.2	25.7	24.7	24.5	23.6
Val RMSE (MW, 1 d.p.)	94.5	41.2	33.2	29.6	32.3	29.0	393.7

Degree with the lowest validation RMSE: $d^\star$ = ___10____

C2. Learning curves. Fit degree 1 and degree 10 on the first m training rows (in file order) and report RMSE on those m rows and on all 300 validation rows. Save the plot as `C2_learning_curves.png`. **[4]**

m	10	15	20	30	45	60
d = 1 train (MW, 1 d.p.)	36.0	49.8	54.1	75.9	81.0	80.5
d = 1 val	92.1	92.7	95.6	93.0	92.9	94.5
d = 10 train	0.0	5.4	15.0	23.0	23.6	24.5
d = 10 val	75447.6	6452.5	7978.6	91.8	34.2	29.0

C3. Now, evaluate on the heatwave days (training-only fits, as in C1). **[3]**

Model	Heatwave RMSE (MW, 1 d.p.)
Degree 3	102.5
Your d^* from C1	10997.3
Degree 15	1099710.2

C4. Related to Physics-informed features. Heating and cooling loads switch on below and above comfort temperatures. Build four features from T: $h = \max(18 - T, 0)$, $c = \max(T - 24, 0)$, h^2 and c^2. Fit `make_pipeline(StandardScaler(), LinearRegression())` on them. Save one plot showing the training data, the heatwave data, your d^* model and this model over 8–47 °C as `C4_extrapolation.png`. **[3]**

	Train RMSE	Val RMSE	Heatwave RMSE
Physics features (MW, 1 d.p.)	24.7	29.8	77.6

C5. Validation picked $d^{\star}$. What went wrong on the heatwave days, and what does a validation set *not* test? In C2, did the degree-10 training error rise as m grew — why? If you had to give the planners one model, which would it be and what warning would you attach? **[6]**

Because the underlying physical relationship is piecewise quadratic with sharp activation thresholds at 18 °C and 24 °C (and a flat comfort deadband in between), a degree-10 polynomial achieved the lowest validation RMSE (29.0 MW) by using high-order terms ($T^7, \dots, T^{10}$) that cancel out inside [8, 40] °C to mimic the flat deadband and sharp elbows. However, once $T >$ 40 °C (test_heatwave spans 41.1–47.0 °C), the T^{10} term grows explosively without bound, predicting up to 29,012 MW at 47 °C and yielding an RMSE of 10,997.3 MW. A validation set drawn from the same temperature distribution (8–40 °C) as the training set only tests **in-distribution interpolation**; it cannot test **out-of-distribution (OOD) extrapolation** to unseen operating regimes.

Yes, the $d = 10$ training RMSE rose monotonically from 0.0 MW at $m = 10$ to 24.5 MW at $m =$ 60. A degree-10 polynomial has $10 + 1 = 11$ free parameters (including the intercept). When $m = 10 \leq 11$, the linear system is under-determined/exact, so the polynomial passes through every noisy training point with zero residual (0.0 MW), memorising measurement noise. As m increases beyond 11 ($m = 15,20, \dots ,60$), the system becomes increasingly over-determined; a single 10th-degree curve can no longer interpolate all noisy observations simultaneously, forcing least squares to compromise across points and causing training RMSE to rise toward the irreducible noise floor ($\approx$ 25 MW).

Recommended model and operational warning for utility planners:

- **Model choice:** Provide the **physics-informed degree-day model** (h, c, h^2, c^2 from C4). It matches the in-range validation accuracy (29.8 MW) while cutting heatwave extrapolation error down to 77.6 MW (versus 10,997.3 MW for $d^{\star} = 10$) because its structural form enforces zero heating load above 18 °C and quadratic cooling growth above 24 °C.
- **Attached warning:** Warn planners that the model was calibrated exclusively on historical days up to 38.3 °C (validation up to 40.0 °C). During extreme heatwaves above 40 °C, air-conditioning saturation (100% duty-cycle limits), thermal derating of distribution transformers, and voltage-dependent load reductions alter the physical load curve (77.6 MW RMSE vs 29.8 MW in-range), so predictions above 40 °C must include an explicit capacity margin and be recalibrated as extreme-temperature metering data is collected

Part D — Ridge, Lasso and Elastic Net [20 marks]

A motor test rig logs 30 signals and the stator-winding temperature rise. Only 60 training runs are available; 500 more are held out for testing. Neighbouring spectral bands of the stator current are correlated with each other, and the `aux_` channels have nothing connected. Use all 30 inputs, standardised, in every model. Use following settings: `RidgeCV(alphas=np.logspace(-3, 3, 61), cv=...)`; `LassoCV(cv=..., max_iter=50000, random_state=0)`; `ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8], cv=..., max_iter=50000, random_state=0)`. The CV splitter as per rule R5.

Own prediction first — fill this in BEFORE you run any code

How many of the 30 signals do you think really drive the temperature rise? Which ones, from your knowledge of motor losses and cooling?

Out of the 30 logged signals, I expect only **4 to 6 physical signals** to genuinely drive the stator-winding temperature rise: **load torque (torque_Nm)** and/or the **fundamental stator current (i_band_01)** (which govern main I^2R copper losses), **low-order harmonic current bands (such as i_band_05 and i_band_07)** (which induce additional eddy-current and stray load heating in the windings and core), and the **cooling system variables (coolant_flow_lpm and coolant_temp_C / ambient_C)** (which govern convective heat removal). The disconnected aux_01–aux_05 channels and high-frequency spectral noise bands have zero physical effect on winding temperature.

D1. Ordinary least squares on all 30 standardised inputs. **[3]**

Train RMSE / test RMSE (K, 3 d.p.)	1.252 K/2.915 K
Five largest |coefficients| (name and value, 2 d.p.)	torque_Nm (+5.66), i_band_05 (+3.26), coolant_flow_lpm (−3.03), i_band_07 (+1.56), i_band_11 (+0.98)

D2. Regularised models. **[4]**

Model	chosen α (3 s.f.)	`l1_ratio`	test RMSE (K, 3 d.p.)	non-zero coefficients
OLS (from D1)	—	—	2.915 K	30
Ridge	0.398	—	2.890 K	30
Lasso	0.121	—	2.373 K	20
Elastic Net	0.107	0.8	2.506 K	23

List the features with non-zero Lasso coefficients:

i_band_02, i_band_05, i_band_06, i_band_07, i_band_08, i_band_09, i_band_10, i_band_11, i_band_12, i_band_13, i_band_14, i_band_15, i_band_17, i_band_18, torque_Nm, ambient_C, coolant_flow_lpm, aux_02, aux_03, aux_05

```
i_band_02       : +0.3109
i_band_05       : +2.7396
i_band_06       : +0.6538
i_band_07       : +1.5923
i_band_08       : +0.4483
i_band_09       : +0.0274
i_band_10       : +0.1649
i_band_11       : +0.5919
i_band_12       : -0.1431
i_band_13       : -0.2807
i_band_14       : -0.1127
i_band_15       : -0.0816
i_band_17       : +0.4875
i_band_18       : -0.2731
torque_Nm       : +5.3888
ambient_C       : +0.0391
coolant_flow_lpm : -3.1579
aux_02          : +0.3146
aux_03          : +0.1456
aux_05          : +0.5548
```

D3. Selection stability: For seeds $s = 0, \ldots, 9$: `idx = np.random.default_rng(s).integers(0, 60, 60)`; fit the scaler and `LassoCV` (same settings) on those training rows; record which features are non-zero. Count, for each feature below, in how many of the 10 fits it was selected. **[5]**

Feature	torque_Nm	i_band_01	i_band_05	i_band_06	i_band_07	i_band_17	coolant_flow_lpm	ambient_C	aux_02	aux_05
Selected (/10)	10	1	10	8	10	10	10	8	10	10

Smallest and largest number of features selected in any of the 10 fits: ____min = 15, max = 26____

D4. The motor designer wants to know which signals drive winding temperature. Write your answer, with how confident you are. Your answer must deal with (i) `i_band_01`, which is strongly

correlated with torque, (ii) the aux_ channels that were selected, and (iii) what cross-validation tunes α for. **[8]**

1. True physical drivers and confidence level:
We can tell the motor designer with **high confidence** that four physical signals dominate stator-winding temperature rise: **load torque (torque_Nm, +ve)**, **coolant flow rate (coolant_flow_lpm, −ve)**, and the **5th and 7th stator-current harmonic bands (i_band_05 and i_band_07, +ve)**. All four have the largest physical magnitudes, consistent signs across OLS and Lasso, and are selected in 10/10 bootstrap resamples. Conversely, we have **low confidence** in the remaining minor spectral bands (i_band_06, i_band_17, etc.) and ambient_C, whose small coefficients fluctuate with collinearity and sample noise.

2. (i) Why i_band_01 is dropped despite driving copper losses:
Physically, fundamental stator current (i_band_01) is nearly proportional to electromagnetic torque (torque_Nm), yielding a strong collinearity ($r \approx 0.86$ on the training set). When two predictors are strongly correlated, the ℓ_1 penalty in Lasso cannot distinguish their individual contributions and arbitrarily zeroes out one of them (i_band_01 is selected only 1/10 times) while assigning the combined fundamental I^2R heating effect to torque_Nm (10/10 times). A zero Lasso weight on a correlated signal does **not** mean it is physically irrelevant.

3. (ii) Why disconnected aux_ channels (aux_02, aux_05) were selected:
With $p = 30$ features and only $m = 60$ training runs ($p/m = 0.5$), pure random noise channels inevitably exhibit spurious sample correlations with the target residuals on the fixed 60-row sample. Because the D3 bootstrap resamples (s = 0..9) draw from the **same** 60 training rows, any noise channel that happened to correlate with the target in those 60 rows (aux_02 and aux_05) gets selected repeatedly (10/10 times).

4. (iii) What cross-validation tunes α for:
Cross-validation tunes α purely to **minimise out-of-sample prediction error (CV MSE)**, **not** to recover the true sparse physical support. Prediction-optimal regularisation chooses a relatively small penalty ($\alpha \approx 0.121$) so it does not over-shrink the large true coefficients (torque_Nm, coolant_flow_lpm, i_band_05, i_band_07). At this small α, the ℓ_1 threshold is too weak to filter out false-positive noise variables (aux_02, aux_03, aux_05), proving that CV-tuned Lasso is a good predictor (2.373 K test RMSE) but an unreliable standalone feature-discovery tool without physical domain knowledge.

Part E — Logistic regression for bearing fault detection [15 marks]

A condition-monitoring system flags bearings as faulty from vibration level and kurtosis. About 15 % of bearings in the data are faulty. A missed fault means an unplanned outage; a false alarm costs an inspection. Use `LogisticRegression()` with defaults. The positive class is `fault = 1`.

Own prediction first — fill this in BEFORE you run any code

If a model is programmed to output always healthy, what would be the accuracy of such a model? Do you expect logistic regression to beat it by a lot?

Since about 15% of the bearings in the dataset are faulty (fault = 1), around 85% are healthy (fault = 0). A trivial classifier programmed to output "always healthy" will achieve **85.0% accuracy** while catching **0% of actual faults**. Because the baseline accuracy is already artificially high (85.0%) due to class imbalance and some incipient faults overlap with healthy vibration levels, I do **not** expect logistic regression to beat 85.0% overall accuracy by a huge percentage margin (at most by 5–10 percentage points, up to ~90–95%), even though its ability to actually detect faulty bearings (recall and precision) will be vastly superior.

E1. Honesty floor and a one-feature model. Fit on `vib_rms_mm_s` only (raw units, not standardised). Evaluate on the test rows at threshold 0.5. **[4]**

Always-healthy test accuracy (%, 1 d.p.)	85.0%
Weight w and bias b (3 d.p.)	w = 2.079, b = −6.834
Decision boundary $v^* = -b/w$ (mm/s, 2 d.p.)	3.29 mm/s
Test accuracy / precision / recall (%, 1 d.p.)	89.2% / 69.2% / 50.0%

E2. Two features (both standardised) at threshold 0.5. Fill in the test confusion matrix. **[3]**

	predicted healthy	**predicted faulty**
actually healthy	99	3
actually faulty	4	14

Overall metrics on the two-feature test set:

accuracy (%)	94.2%
precision (%)	82.4%
recall (%)	77.8%

E3. Moving the threshold. Maintenance wants recall of at least 80 %. Using the *validation* rows and thresholds `np.round(np.arange(0.05, 0.951, 0.05), 2)`, pick the largest threshold τ that reaches validation recall ≥ 0.80. Then report test performance at τ. **[4]**

Chosen τ	0.10
Highest validation recall reachable on this grid (%)	83.3%
Test recall / precision at τ (%, 1 d.p.)	83.3% / 65.2%
Test false-alarm rate FP/(FP+TN) at τ (%, 1 d.p.)	7.8%
P(fault) for a bearing with v = 2.6 mm/s, kurtosis 4.2 (3 d.p.)	0.693

E4. Why can no threshold reach 95 % recall here? What would actually raise it? Which two numbers would you put in front of the maintenance manager, and why not accuracy? **[4]**

Out of the 18 faulty bearings in the validation set (and 18 in the test set), **3 faulty bearings** have predicted probabilities strictly below the lowest grid threshold $\tau = 0.05$ (e.g., validation rows with $v_{\text{rms}} = 3.58$ mm/s,kurtosis $= 1.35 \Rightarrow P = 0.005$ and $v_{\text{rms}} = 3.41$ mm/s,kurtosis $= 1.68 \Rightarrow P = 0.009$, plus an incipient fault at $v_{\text{rms}} = 2.27$ mm/s,kurtosis $= 3.15 \Rightarrow P = 0.049$). Physically, localized spalling creates high impulsive kurtosis (> 3.5), whereas advanced distributed wear or unbalance produces high RMS vibration with **low, sub-Gaussian kurtosis** (< 2.0). Because logistic regression fits a **single linear hyperplane** ($w_1 z_{\text{vib}} + w_2 z_{\text{kurt}} + b = 0$) with a large positive weight on kurtosis ($w_2 = +2.42$), a negative standardized kurtosis cancels out high RMS vibration, pushing those worn bearings deep into the healthy half-space ($P < 0.05$) and capping validation recall at $15/18 = 83.3\%$.

Non-linear / interaction features or rule-based OR boundaries: Adding polynomial/quadratic features (v_{rms}^2, |kurtosis $- 3.0$|, or an RBF kernel / decision tree) so that **either** high RMS vibration **or** abnormal kurtosis triggers a fault alarm without cancelling each other out.
Frequency-domain condition-monitoring features: Extracting envelope-spectrum (high-frequency demodulated) amplitudes at characteristic bearing defect frequencies (**BPFO, BPFI, BSF, FTF**) and temperature/acoustic trends to separate incipient faults from normal baseline noise.

Two numbers to report: Present **Recall (83.3%, or missed-fault rate 16.7% = 3 missed faults out of 18)** alongside **Precision (65.2%) or False-Alarm Rate (7.8% = 8 false inspections per 102 healthy bearings)** at $\tau = 0.10$. These two numbers directly quantify the operational trade-off between **unplanned outage risk** (missed faults) and **wasted inspection labour** (false alarms).
Why not accuracy: Because 85.0% of bearings are healthy, a broken sensor or dummy script that predicts "always healthy" achieves **85.0% accuracy** while missing **100% of faults**. Overall accuracy is dominated by true negatives (TN) and masks catastrophic false negatives (FN).

Part F — Softmax regression for power-quality events [10 marks]

A power-quality recorder labels each event as one of *normal*, *sag*, *swell*, *harmonic* from the event's RMS voltage and voltage THD. Use `make_pipeline(StandardScaler(), LogisticRegression(C=10, max_iter=2000))`; with more than two classes this is softmax (multinomial) regression.

F1. Fit on the training rows; fill in the test confusion matrix (rows = true, columns = predicted) and per-class recall. Save a scatter plot of the training data with the decision regions as `F1_regions.png`. **[3]**

true \ pred	normal	sag	swell	harmonic	recall (%, 1 d.p.)
normal	90	0	0	0	100.0%
sag	0	44	0	1	97.8%
swell	0	0	36	0	100.0%
harmonic	3	0	0	36	92.3%
Overall test accuracy (%, 1 d.p.)					98.1%

F2. Predicted class probabilities (3 d.p.). **[2]**

Event	normal	sag	swell	harmonic
0.97 p.u., THD 5.0 %	0.420	0.110	0.000	0.469
0.84 p.u., THD 8.0 %	0.000	0.998	0.000	0.002

F3. How many events in the whole file are labelled *sag* but have THD above 6 %? **____19 (11 in train and 8 in test)._** Look at them on your plot. Is a single label the whole truth for these events? What would you build instead of softmax, and how many outputs would it have? What should an operator do with the first event in F2? **[5]**

No. Looking at the upper-left quadrant of F1_regions.png ($V_{\text{rms}} \in [0.76,0.90]$ p.u., THD $\in$ [6.6%, 11.9%]), those 19 points experience **simultaneous voltage sag and harmonic distortion** (for example, a feeder fault, transformer energisation, or large motor start occurring alongside non-linear power-electronic loads). Because the recorder forced a single mutually exclusive label (sag), softmax learned a tilted linear boundary that assigns the second event in F2 (0.84 p.u., THD 8.0%) a 0.998 probability of sag and only 0.002 probability of harmonic, completely hiding a severe 8.0% harmonic violation.

Replace mutually exclusive softmax with a **multi-label binary classifier** (independent logistic/sigmoid classifiers via MultiOutputClassifier(LogisticRegression()), or decoupled physical heads) with **3 independent binary sigmoid outputs**: **[is_sag, is_swell, is_harmonic]** (where normal is the state when all three flags are 0, or **2 decoupled outputs**: a 3-class softmax for voltage magnitude [normal_V, sag, swell] plus 1 independent binary sigmoid for [is_harmonic]). Because each sigmoid output $P(y_k = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}_k^{\top}\mathbf{x} + b_k)$ is evaluated independently rather than constrained to sum to 1, the event (0.84 p.u., THD 8.0%) will simultaneously flag $P(\text{sag}) \approx 1.0$ **and** $P(\text{harmonic}) \approx 1.0$.

At 0.97 p.u., THD 5.0%, the softmax probabilities are split across three classes (harmonic = 0.469, normal = 0.420, sag = 0.110), with no class reaching even 50% confidence because the point sits right on the triple decision boundary (at the IEEE 519 5.0% THD limit and near the 0.90 p.u. sag boundary). Rather than trusting the raw argmax (harmonic) to trigger automated switching or dismissing it as normal, the monitoring system should **abstain from automated control action and raise a low-confidence "Borderline / Watch" advisory**, prompting the operator to inspect waveform trend logs to verify whether the 5.0% THD is a brief transient or a sustained harmonic resonance requiring filter/capacitor-bank adjustment.

Part G — Caught GenAI being wrong **[required]**

Describe one place in this homework where a GenAI tool gave you something wrong, misleading, or unqualified (code, an explanation, or a number). Quote what it said, say how you found out, and what the correct version is. If it never misled you, describe the claim you checked most carefully, and how.

1. What the GenAI tool said:
When I asked how to get the temperature coefficient α from the linear fit $R = \theta_0 + \theta_1 T$ in Part A2, the GenAI tool wrote:
"Since $R(T) = R_0(1 + \alpha T) = R_0 + R_0 \alpha T$, simply compute alpha = theta1 / theta0."
Using my fitted values, that formula gave $\alpha = 0.00333176/0.783905 = 0.004250\ ^\circ\mathrm{C}^{-1}$.
2. How I found out it was wrong:
The value $0.004250\ ^\circ\mathrm{C}^{-1}$ was 8.1% higher than the standard copper handbook value of $0.00393\ ^\circ\mathrm{C}^{-1}$, even though the bench data had very low noise. Re-reading the formula on the task sheet showed that the reference temperature is 20 ˚C, so $R(T) = R_{20}[1 + \alpha(T - 20)]$, whereas θ_0 is the resistance at 0 ˚C, not 20 ˚C.
3. What the correct version is:
First compute the resistance at 20 ˚C as $R_{20} = \theta_0 + 20\theta_1 = 0.8505\ \Omega$, and then divide the slope θ_1 by R_{20}:
alpha = theta1 / R20
which gives the correct value $\alpha = 0.003917\ ^\circ\mathrm{C}^{-1}$ (within 0.3% of the copper handbook value).